

Assignment 2: TinyGPT

Implementing a Character-Level Generative Pre-trained Transformer

CSE 676 – Deep Learning
Spring 2026

Assigned: April 23, 2026 Due: May 14, 2026 at 11:59 PM

Contents

1 Overview	3
2 Background: Language Models and Autoregression	3
2.1 What is a Language Model?	3
2.2 Character-Level Tokenisation	4
2.3 Training Objective: Next-Character Prediction	4
3 Background: The GPT Architecture	4
3.1 High-Level Overview	4
3.2 Token and Positional Embeddings	4
3.3 Causal (Masked) Self-Attention	5
3.4 The GPT Block (Transformer Decoder Layer)	6
3.5 Layer Normalisation	6
3.6 Residual Connections and Why They Matter	7
3.7 Text Generation	7
4 Code Template: gpt_template.py	8
4.1 File Structure	8
4.2 Step-by-Step Usage	8
5 Tasks	9
5.1 Part 1: Implementation (60 points)	9
5.2 Part 2: Written Report (40 points)	10
6 JSON Log Format	11
7 Checkpoint Format	11
8 Using Git for This Assignment	12
8.1 Initial Setup (Day 1)	12
8.2 Recommended Commit Workflow	12
8.3 Verifying Your Git History	12

9 Submission Format	13
10 Grading Summary	13
11 Academic Integrity	14
12 Tips and Common Pitfalls	14

1 Overview

In this assignment you will build your own large language model (LLM) from scratch. You will implement **TinyGPT**: a character-level, decoder-only transformer that reads the complete works of Shakespeare and learns to imitate his style. The model trains on raw characters (no tokenizer, no pre-built vocabulary) and discovers the structure of language entirely from data. By the end, you will be able to provide a prompt such as `HAMLET:\n` and watch your model continue it with Shakespeare-ish dialogue.

More concretely, you will implement **TinyGPT** as a character-level, decoder-only transformer language model inspired by the original GPT paper (Radford et al., 2018). You will train the model on the TinyShakespeare corpus and generate text samples at multiple training checkpoints.

This assignment has two components:

1. **Implementation (60 pts)** – Implement six components inside `gpt_template.py`, including the causal self-attention mechanism, the transformer block, the full GPT model, the training loop, and the text-generation function. Evaluated automatically.
2. **Written Report (40 pts)** – A PDF report (max 4 pages) discussing your training results and generated samples. Graded manually by the teaching assistants.

The model is intentionally small ($\approx 144\text{K}$ parameters) and trains on the CPU in **3–5 minutes** on a modern laptop.

Important

Your submission must contain a `.git` directory with at least **5 commits** spread across at least **2 different calendar days**. A dedicated script (`check-git.py`) will verify this automatically. Failing this check results in a **15-point deduction**.
Your submission must also include **report.pdf**. Missing the report means forfeiting the 40 report points.

2 Background: Language Models and Autoregression

2.1 What is a Language Model?

A **language model** assigns a probability to a sequence of tokens. Given a sequence $(x_1, x_2, \dots, x_T)$, the chain rule of probability factorises the joint distribution as a product of conditional probabilities:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, x_2, \dots, x_{t-1}). \quad (1)$$

This is the *autoregressive* (or *causal*) factorisation. At each position t , the model conditions on all past tokens and predicts the *next* token. Because the model must not look into the future when predicting x_t , its attention mechanism must be *causal*: position t may attend only to positions $1, 2, \dots, t$.

2.2 Character-Level Tokenisation

In this assignment we use **character-level** tokenisation: the vocabulary $\mathcal{V}$ is the set of all unique characters in the corpus (65 for TinyShakespeare), and each character is mapped to an integer index. This is the simplest possible tokenisation scheme and requires no external tokeniser.

raw text					
H	A	M	L	E	T
7	0	12	11	4	19
integer indices					

2.3 Training Objective: Next-Character Prediction

Given an input context $\mathbf{x} = (x_1, \dots, x_T)$, the model produces logits $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$ for every position t . The training loss is the average cross-entropy over all positions:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log P(x_{t+1} \mid x_1, \dots, x_t) = -\frac{1}{T} \sum_{t=1}^T \log \text{softmax}(\mathbf{z}_t)[x_{t+1}]. \quad (2)$$

During training, the target at position t is the *next* character x_{t+1} .

3 Background: The GPT Architecture

3.1 High-Level Overview

GPT (Generative Pre-trained Transformer) is a decoder-only transformer introduced by Radford et al. (2018). Unlike the full encoder–decoder Transformer of Vaswani et al. (2017) [3], GPT uses only the decoder stack and removes the encoder-to-decoder cross-attention. The architecture is described informally as “a bunch of stacked causal self-attention layers with feed-forward sub-layers and layer normalisation”.

Figure 1 shows the complete pipeline from an input token sequence to output logits.

3.2 Token and Positional Embeddings

Token embedding. Each integer token index $x_t \in \{0, \dots, |\mathcal{V}| - 1\}$ is mapped to a dense vector via a learned embedding matrix $\mathbf{W}_e \in \mathbb{R}^{|\mathcal{V}| \times D}$:

$$\mathbf{e}_t = \mathbf{W}_e[x_t] \in \mathbb{R}^D.$$

Positional embedding. Self-attention has no built-in notion of sequence order (it is permutation-invariant). To inform the model about the position of each token, a learned positional embedding $\mathbf{W}_p \in \mathbb{R}^{T_{\max} \times D}$ is added:

$$\mathbf{h}_t^{(0)} = \mathbf{e}_t + \mathbf{W}_p[t].$$

The input to the first transformer block is the matrix $\mathbf{H}^{(0)} \in \mathbb{R}^{T \times D}$ of these summed embeddings, optionally followed by dropout.

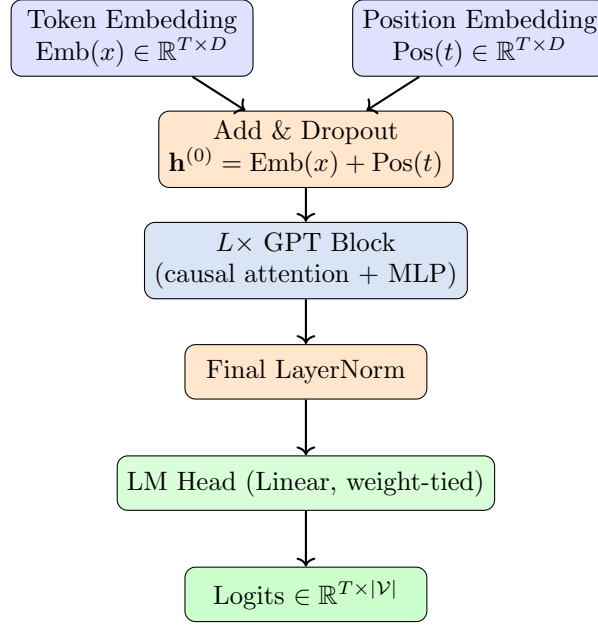


Figure 1: TinyGPT architecture. Token and position embeddings are summed, then passed through L GPT blocks (each a causal self-attention layer followed by a feed-forward network), a final layer normalisation, and a weight-tied linear head that projects to vocabulary logits.

Weight tying. The output linear head projects the final hidden state back to vocabulary logits. In GPT-2, this head *shares its weight matrix with the token embedding*:

$$\text{logits}_t = \mathbf{W}_e \mathbf{h}_t^{(L)}$$

Weight tying reduces the parameter count by $|\mathcal{V}| \times D$ and typically improves perplexity, because the model is encouraged to produce final representations that are close to the embedding vectors of probable next tokens.

3.3 Causal (Masked) Self-Attention

The central operation of GPT is **multi-head causal self-attention**. For a single head with queries $\mathbf{Q}$, keys $\mathbf{K}$, values $\mathbf{V} \in \mathbb{R}^{T \times d_k}$, the attention output is:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}, \quad (3)$$

where $\mathbf{M}$ is the **causal mask**:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases}$$

Positions masked to $-\infty$ contribute zero weight after the softmax, so token i can attend only to tokens at positions $\leq i$. Figure 2 illustrates this for a sequence of length 6.

Multi-head attention. With h heads and head dimension $d_k = D/h$, the token sequence $\mathbf{H} \in \mathbb{R}^{T \times D}$ is projected separately for each head:

$$\mathbf{Q}^{(i)} = \mathbf{H}\mathbf{W}_Q^{(i)}, \quad \mathbf{K}^{(i)} = \mathbf{H}\mathbf{W}_K^{(i)}, \quad \mathbf{V}^{(i)} = \mathbf{H}\mathbf{W}_V^{(i)}, \quad \mathbf{W} \in \mathbb{R}^{D \times d_k}.$$

	k_1	k_2	k_3	k_4	k_5	k_6	
q_1	a_{11}	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	attend (causal)
q_2	a_{21}	a_{22}	$-\infty$	$-\infty$	$-\infty$	$-\infty$	masked ($-\infty$)
q_3	a_{31}	a_{32}	a_{33}	$-\infty$	$-\infty$	$-\infty$	
q_4	a_{41}	a_{42}	a_{43}	a_{44}	$-\infty$	$-\infty$	
q_5	a_{51}	a_{52}	a_{53}	a_{54}	a_{55}	$-\infty$	
q_6	a_{61}	a_{62}	a_{63}	a_{64}	a_{65}	a_{66}	

Figure 2: Causal attention mask for a sequence of length $T = 6$. Query position i (row) can attend to key position j (column) only if $j \leq i$. The upper-triangular entries are set to $-\infty$ before the softmax, preventing the model from “seeing the future.”

In practice, the three projections are batched into a single matrix multiply using a combined weight $\mathbf{W}_{qkv} \in \mathbb{R}^{D \times 3D}$:

$$[\mathbf{Q} \parallel \mathbf{K} \parallel \mathbf{V}] = \mathbf{H} \mathbf{W}_{qkv}.$$

The outputs of all h heads are concatenated and projected back:

$$\text{MHSA}(\mathbf{H}) = \text{concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O, \quad \mathbf{W}_O \in \mathbb{R}^{D \times D}.$$

3.4 The GPT Block (Transformer Decoder Layer)

Each of the L GPT blocks applies causal self-attention and a feed-forward network (**MLP**) as residual sub-layers. GPT-2 (and TinyGPT) use **pre-norm**: the layer normalisation is applied *before* each sub-layer rather than after:

$$\mathbf{h}' = \mathbf{h} + \text{MHSA}(\text{LN}(\mathbf{h})), \tag{4}$$

$$\mathbf{h}'' = \mathbf{h}' + \text{MLP}(\text{LN}(\mathbf{h}')). \tag{5}$$

The MLP is a two-layer feed-forward network with GELU activation:

$$\text{MLP}(\mathbf{x}) = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \mathbf{x}), \quad \mathbf{W}_1 \in \mathbb{R}^{D_{\text{mlp}} \times D}, \quad \mathbf{W}_2 \in \mathbb{R}^{D \times D_{\text{mlp}}}.$$

The hidden dimension D_{mlp} is typically $2 \times$ to $4 \times$ the embedding dimension D .

Figure 3 shows the data flow within a single GPT block.

3.5 Layer Normalisation

Layer Normalisation [4] standardises each token’s feature vector independently across the embedding dimension:

$$\text{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma + \epsilon} + \beta, \quad \mu = \frac{1}{D} \sum_{d=1}^D x_d, \quad \sigma = \sqrt{\frac{1}{D} \sum_{d=1}^D (x_d - \mu)^2},$$

where γ and β are learned scale and bias parameters and ϵ prevents division by zero. Unlike Batch Normalisation, LN operates on each example independently, making it suitable for variable-length sequences and small batch sizes.

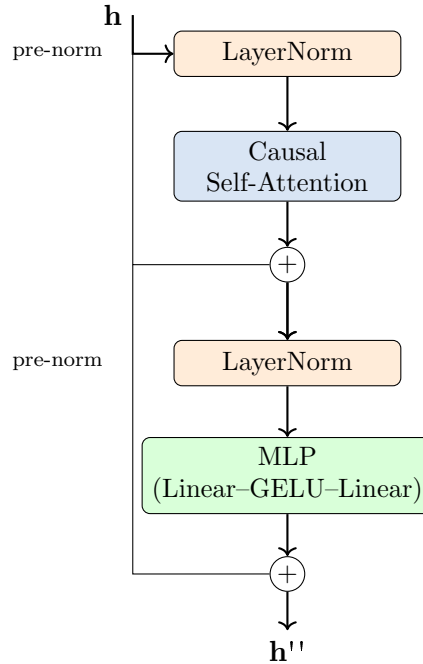


Figure 3: A single GPT block (pre-norm variant). The LayerNorm is applied *before* each sub-layer. Residual connections (skip connections) add the original input back so that gradients flow freely even through many layers.

3.6 Residual Connections and Why They Matter

Both sub-layers in a GPT block are wrapped in a residual (skip) connection: $\mathbf{h}' = \mathbf{h} + F(\mathbf{h})$. Residuals serve two purposes:

1. **Gradient highway.** The gradient of the loss with respect to layer ℓ 's input contains an additive term from the residual path that bypasses the sub-layer, reducing the vanishing gradient problem.
2. **Identity initialisation.** At the start of training, if the sub-layer outputs are near zero, the block behaves like an identity function. This makes deep stacks easier to optimise.

3.7 Text Generation

Once the model is trained, new text is generated **autoregressively**:

1. Encode the prompt string as a tensor of token indices.
2. Forward pass the context (capped at `block_size` tokens) through the model.
3. Extract the logits at the *last* position: $\mathbf{z}_T \in \mathbb{R}^{|\mathcal{V}|}$.
4. Apply the *temperature-scaled* softmax:

$$P(x_{T+1} = k) = \frac{\exp(z_k/\tau)}{\sum_j \exp(z_j/\tau)},$$

where $\tau > 0$ is the temperature parameter.

5. Sample the next token $x_{T+1} \sim P(\cdot)$.

6. Append x_{T+1} to the context and repeat from step 2.

Concept

Temperature τ and diversity:

- $\tau \rightarrow 0$: the distribution collapses to a one-hot at the most-likely token (greedy decoding). Produces repetitive, low-diversity text.
- $\tau = 1$: unmodified softmax. Standard stochastic sampling.
- $\tau > 1$: distribution becomes more uniform; text is more diverse but potentially less coherent.

Temperature is one reason LLM outputs are non-deterministic: by shaping the probability distribution before sampling, it controls how much randomness enters each token choice.

4 Code Template: gpt_template.py

All your work lives in a single file: `gpt_template.py`. Do not rename it. Do not modify function or class signatures. The autograder imports the file and calls specific functions by name.

4.1 File Structure

```
gpt_template.py
|
|-- SEED = 42                # fixed random seed (do not modify)
|
|-- set_all_seeds(seed)      # seeds Python / NumPy / PyTorch
|
|-- CharDataset              # provided (do not modify)
|
|-- get_dataset()            # TODO 1.1 (you implement)
|-- CausalSelfAttention      # TODO 1.2 (you implement)
|-- GPTBlock                 # TODO 1.3 (you implement)
|-- GPT                      # TODO 1.4 (you implement)
|-- train_model()            # TODO 1.5 (you implement)
|-- generate()               # TODO 1.6 (you implement)
|
|-- run_all()                # provided -- trains and generates text
|
|-- if __name__ == "__main__":
|     run_all()
```

4.2 Step-by-Step Usage

Step 1: Install dependencies.

```
pip install torch>=2.0 numpy
```

No GPU is needed. TinyShakespeare is downloaded automatically on first run.

Step 2: Run everything. After implementing all six stubs:

```
python gpt_template.py --mode all
```


This trains the baseline model, saves checkpoints, and generates text samples. Outputs: `training_log.json`, `generated_text.json`, `checkpoints/`.

Step 3: Write your report. Inspect the generated JSON files and write a PDF report (`report.pdf`) as described in Section 5.2.

Note

Baseline configuration: `block_size` 128, `embed_dim` 64, 4 attention heads, 4 GPT blocks, MLP dim 128, dropout 0.1, `batch_size` 64, lr 3×10^{-4} , 5 epochs of 200 gradient steps each. This produces roughly **144 000 parameters** and trains in **3–5 minutes** on a laptop CPU.

5 Tasks

5.1 Part 1: Implementation (60 points)

Implement the six stubs in `gpt_template.py`.

1.1 `get_dataset` (5 pts)

Download TinyShakespeare to `data/input.txt` (use `urllib.request.urlretrieve` with `DATA_URL`). Build a character vocabulary: `sorted(set(text))`. Encode the full text as a `LongTensor`. Split 90 % for training and 10 % for validation. Return two `CharDataset` instances, `vocab_size`, and the `stoi/itos` dicts.

1.2 `CausalSelfAttention` (20 pts)

Implement multi-head causal self-attention from scratch. **Do not use** `nn.MultiheadAttention`.

In `__init__`:

- Assert `embed_dim % num_heads == 0`.
- Store `self.num_heads`, `self.head_dim`, `self.embed_dim`.
- Create `self.qkv = nn.Linear(embed_dim, 3*embed_dim, bias=False)`.
- Create `self.out_proj = nn.Linear(embed_dim, embed_dim, bias=False)`.
- Create `self.attn_drop = nn.Dropout(dropout)`.
- Create `self.resid_drop = nn.Dropout(dropout)`.
- Build the causal mask: `torch.tril(torch.ones(block_size, block_size))`. Register it as a buffer named "mask" with shape `(1, 1, block_size, block_size)`.

In `forward(x)`:

- Project `x` through `self.qkv`; split to get `q`, `k`, `v` (each `(B, T, D)`).
- Reshape each to `(B, num_heads, T, head_dim)`.
- Compute $\mathbf{QK}^\top / \sqrt{d_k}$, mask future positions (`masked_fill` where `mask==0` with `-inf`), softmax, apply `self.attn_drop`.
- Multiply by `v`; reshape to `(B, T, D)`.
- Apply `self.out_proj` and `self.resid_drop`.

1.3 `GPTBlock` (10 pts)

Implement the pre-norm transformer block (equations 4 and 5). Required sub-module names:

`norm1`, `attn`, `norm2`, `mlp`. The MLP must be an `nn.Sequential`: `Linear` $\rightarrow$ `GELU` $\rightarrow$ `Linear` $\rightarrow$ `Dropout`.

1.4 GPT (10 pts)

Assemble the full model: `token_embedding`, `pos_embedding`, `drop`, `blocks` (`nn.ModuleList`), `norm`, and `head`. **Tie weights:** after creating `self.head`, set `self.head.weight = self.token_embedding`. The `forward` method must return logits of shape `(B, T, vocab_size)`.

1.5 train_model (10 pts)

Implement the full training loop. Requirements:

- **Optimizer:** `AdamW`, `weight_decay=1e-2`.
- **LR scheduler:** `CosineAnnealingLR`, `T_max = epochs`.
- **Loss:** cross-entropy over all positions in the batch (flatten to `(B*T, V)` before `F.cross_entropy`).
- **Gradient clipping:** `clip_grad_norm(model.parameters(), 1.0)`.
- **Steps per epoch:** run exactly `config["steps_per_epoch"]` gradient updates, sampling batches from a shuffled `DataLoader`.
- **Validation:** after each epoch, evaluate on up to 50 batches (no gradients).
- **Checkpoints:** save at epochs in `CHECKPOINT_EPOCHS`.
- **JSON log:** write `training_log.json` (schema in Section 6).

1.6 generate (5 pts)

Implement autoregressive text generation as described in Section 3.7. The function must:

- Set the model to `eval()` mode.
- Encode the prompt using `stoi`.
- Run `max_new_tokens` sampling steps.
- Decode and return the full string (prompt + generated).

5.2 Part 2: Written Report (40 points)

After running `python gpt_template.py --mode all`, you will have `training_log.json` (with 5 epochs of loss data) and `generated_text.json` (with text samples at temperatures 0.5, 1.0, and 1.5 for each checkpoint epoch).

Write a PDF report (`report.pdf`, **maximum 4 pages**) that addresses all four of the following sections.

R.1 Training Curve (10 pts)

Plot the training loss and validation loss versus epoch from `training_log.json`. Comment on whether the model is overfitting, underfitting, or learning normally. Quote specific values (e.g., “final val_loss dropped from X at epoch 1 to Y at epoch 5”).

R.2 Generated Text Analysis (15 pts)

Show the generated text from checkpoints at epochs 1, 3, and 5 (temperature 1.0). How does the quality of the generated text change as training progresses? At what epoch does the model begin to produce recognisable English words? Full sentences? Quote specific excerpts to support your claims.

R.3 Temperature Analysis (10 pts)

For the epoch-5 checkpoint, compare the text generated at temperatures 0.5, 1.0, and 1.5.

Explain, with reference to the theory in Section 3.7, why higher temperature produces more diverse (but potentially less coherent) output. Provide specific excerpts.

R.4 Reflection (5 pts)

What has TinyGPT learned about English text? Consider: character n-gram statistics, word boundaries, capitalisation rules, and any higher-level structure (e.g., dialogue formatting in Shakespeare). What are the limitations of this model, and how might a larger model (more layers, wider embedding, or subword tokenisation) overcome them?

6 JSON Log Format

training_log.json must conform to this exact schema:

```
{
  "seed": 42,
  "config": {
    "block_size": 128,
    "embed_dim": 64,
    "num_heads": 4,
    "num_layers": 4,
    "mlp_dim": 128,
    "dropout": 0.1,
    "lr": 0.0003,
    "batch_size": 64,
    "epochs": 5,
    "steps_per_epoch": 200
  },
  "history": [
    {
      "epoch": 1,
      "train_loss": 2.8512,
      "val_loss": 2.7634,
      "epoch_time_sec": 14.2
    },
    ...
  ],
  "final_val_loss": 1.8341,
  "total_params": 144128
}
```

Note

Round all floating-point values to 4 decimal places. The autograder checks the JSON schema structure exactly.

7 Checkpoint Format

Save checkpoints using:

```
1 torch.save({
2     "model_state_dict": model.state_dict(),
3     "config": {k: config[k] for k in
```

```

4         ["block_size", "embed_dim", "num_heads",
5          "num_layers", "mlp_dim", "dropout"]},
6     "epoch": epoch,
7 }, os.path.join(checkpoint_dir, f"gpt_epoch_{epoch}.pt"))

```

Checkpoints are saved at epochs 1, 3, and 5 (defined in `CHECKPOINT_EPOCHS`). The autograder loads these to run the `generate` test.

8 Using Git for This Assignment

Important

Your submission zip must include the `.git` directory. `check-git.py` will verify that you have at least **5 commits** on at least **2 different calendar days**. A single commit at the deadline is a **15-point deduction**.

8.1 Initial Setup (Day 1)

```

cd UBNumber/
git init
git add gpt_template.py
git commit -m "initial commit"

```

8.2 Recommended Commit Workflow

Commit 1. (*Day 1*) Implement `get_dataset` and `CausalSelfAttention`. Verify forward pass shape.

```
git commit -m "implement get_dataset and CausalSelfAttention"
```

Commit 2. (*Day 2–3*) Implement `GPTBlock` and `GPT`. Test a forward pass on random inputs.

```
git commit -m "implement GPTBlock and GPT model"
```

Commit 3. (*Day 3–4*) Implement `train_model` and `generate`. Run `python gpt_template.py --mode all`.

```

git add gpt_template.py training_log.json generated_text.json
      checkpoints/
git commit -m "complete Part 1; final val_loss = X.XXX"

```

Commit 4. (*Day 4–5*) Write and finalise the report.

```

git add report.pdf
git commit -m "add written report"

```

8.3 Verifying Your Git History

```
python check-git.py UBNumber/
```

Expected output:

```
Commits found: 6
Unique calendar days: 3
PASS: git history requirement satisfied.
```

9 Submission Format

Submit a single zip file named `UBNumber.zip`. When unzipped, it must contain:

```
UBNumber/
  .git/                                <- must be present; see Section 8
  gpt_template.py
  training_log.json
  generated_text.json
  report.pdf                          <- written report (max 4 pages)
  checkpoints/
    gpt_epoch_1.pt
    gpt_epoch_3.pt
    gpt_epoch_5.pt
```

Create the zip with:

```
cd ..
zip -r UBNumber.zip UBNumber/
```

Verify `.git` is included:

```
unzip -l UBNumber.zip | grep ".git/"
```

10 Grading Summary

Part	Component	Points
1	get_dataset	5
1	CausalSelfAttention	20
1	GPTBlock	10
1	GPT	10
1	train_model (log schema + ckpts)	10
1	generate	5
2	Report: Training Curve	10
2	Report: Generated Text Analysis	15
2	Report: Temperature Analysis	10
2	Report: Reflection	5
<i>Deduction: Git history violation</i>		−15
<i>Deduction: Modified function signatures</i>		−10
Total		100

11 Academic Integrity

You may reference: the original GPT papers, the Transformer paper, PyTorch documentation, and course lecture notes. You may **not** copy code from existing GPT implementations (nanoGPT, HuggingFace Transformers, minGPT, GitHub repositories, etc.).

Use of large language models (LLMs) to generate implementation code is **not permitted**. You are expected to understand and be able to explain every line you submit.

12 Tips and Common Pitfalls

- **Set STUDENT_ID first.** Every random outcome in the assignment depends on it.
- **Check tensor shapes at every step.** The causal mask must have shape $(1, 1, \text{block_size}, \text{block_size})$. After the QKV split, `q.shape` must be $(B, \text{num_heads}, T, \text{head_dim})$.
- **Mask before softmax, not after.** The mask must be applied to the raw attention logits $(\mathbf{QK}^\top / \sqrt{d_k})$ *before* calling `softmax`, not to the resulting probabilities.
- **Weight tying.** Set `self.head.weight = self.token_embedding.weight` *after* creating both layers. Verify with `assert model.head.weight is model.token_embedding.weight`.
- **Crop context during generation.** When the growing context exceeds `block_size`, crop to the last `block_size` tokens before feeding the model: `ctx_tensor[:, -model.block_size:]`.

References

- [1] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI, 2018.
- [2] A. Radford et al., “Language Models are Unsupervised Multitask Learners,” OpenAI, 2019. (GPT-2)
- [3] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [4] J. L. Ba, J. R. Kiros, G. E. Hinton, “Layer Normalization,” *arXiv*, 2016.
- [5] A. Karpathy, “The Unreasonable Effectiveness of Recurrent Neural Networks,” blog post, 2015. (Source of TinyShakespeare.)